



Session HTTP

LARAVEL



INTRODUCTION

INTRODUCTION

Les sessions permettent de stocker des informations sur l'utilisateur à travers plusieurs requêtes HTTP. Ces informations sur l'utilisateur sont généralement placées dans un magasin persistant auquel on peut accéder lors de requêtes ultérieures.

Laravel est livré avec une variété de backends de session accessibles via une API expressive et unifiée.

CONFIGURATION

Le fichier de configuration de la session de votre application est stocké à l'adresse **config/session.php**. Veuillez à examiner les options disponibles dans ce fichier.

Par défaut, Laravel est configuré pour utiliser le pilote de session **file**, ce qui convient à de nombreuses applications. Si votre application doit être répartie sur plusieurs serveurs Web, vous devez choisir un magasin centralisé auquel tous les serveurs peuvent accéder, comme **Redis** ou une **base de données**.

LE PILOTE DATABASE

Lorsque vous utilisez le pilote de session **database**, vous devez créer une table pour contenir les enregistrements de session. Vous trouverez ci-dessous un exemple de déclaration de schéma pour la table :

```
Schema::create('sessions', function ($table) {  
    $table->string('id')->primary();  
    $table->foreignId('user_id')->nullable()->index();  
    $table->string('ip_address', 45)->nullable();  
    $table->text('user_agent')->nullable();  
    $table->text('payload');  
    $table->integer('last_activity')->index();  
});
```

LE PILOTE DATABASE

Vous pouvez utiliser la commande **Artisan session:table** pour générer cette migration.

```
php artisan session:table
```

```
php artisan migrate
```


The background is a white canvas covered with numerous colorful paint splatters and dots in shades of orange, red, purple, blue, and green. A large, semi-transparent hexagonal frame with a thin dark border is centered on the page. Inside this frame, the text "INTERAGIR AVEC LA SESSION" is written in a bold, black, serif font, arranged in two lines.

INTERAGIR AVEC LA SESSION

RÉCUPÉRATION DES DONNÉES

Il y a deux façons principales de travailler avec les données de session dans Laravel : la fonction **session** et via une instance de requête. Tout d'abord, examinons l'accès à la session via une instance de requête, qui peut être indiquée par un type sur une fermeture de route ou une méthode de contrôleur.

```
public function show(Request $request, $id)
{
    $value = $request->session()->get('key');
}
```


RÉCUPÉRATION DES DONNÉES

Lorsque vous récupérez un élément dans la session, vous pouvez également passer une valeur par défaut comme deuxième argument de la méthode **get**. Cette valeur par défaut sera renvoyée si la clé spécifiée n'existe pas dans la session.

```
$value = $request->session()->get('key', 'default');  
  
$value = $request->session()->get('key', function () {  
    return 'default';  
});
```

LE HELPER SESSION()

Vous pouvez utiliser la fonction **session** pour récupérer et stocker des données dans la session. Lorsque le Helper **session** est appelée avec un seul argument de type chaîne, elle renvoie la valeur de la clé de session. Lorsqu'il est appelée avec un tableau de paires clé/valeur, ces valeurs seront stockées dans la session :

```
// Récupérer un élément de données de la session...  
$value = session('key');  
// Spécifier une valeur par défaut...  
$value = session('key', 'default');  
// Stocker un élément de données dans la session...  
session(['key' => 'value']);
```

RÉCUPÉRER TOUTES LES DONNÉES DE LA SESSION

Si vous souhaitez récupérer toutes les données de la session, vous pouvez utiliser la méthode **all** :

```
$data = $request->session()->all();
```

PRÉSENCE DES DONNÉES DE LA SESSION

Pour déterminer si un élément est présent dans la session, vous pouvez utiliser la méthode **has**. La méthode **has** renvoie **true** si l'élément est présent et n'est pas nul :

```
if ($request->session()->has('users')) {  
    //  
}
```

PRÉSENCE DES DONNÉES DE LA SESSION

Pour déterminer si un élément est présent dans la session, même si sa valeur est nulle, vous pouvez utiliser la méthode **exists** :

```
if ($request->session()->exists('users')) {  
    //  
}
```


PRÉSENCE DES DONNÉES DE LA SESSION

Pour déterminer si un élément **n'est pas présent** dans la session, vous pouvez utiliser la méthode **missing**. La méthode **missing** renvoie la valeur **true** si l'élément n'est pas présent :

```
if ($request->session()->missing('users')) {  
    //  
}
```

STOCKAGE DES DONNÉES

Pour stocker des données dans la session, vous utiliserez généralement la méthode **put** de l'instance de requête ou le Helper global **session()** :

```
// Via une instance de demande...  
$request->session()->put('key', 'value');  
  
// Via le Helper globale "session" ...  
session(['key' => 'value']);
```

POUSSER LES VALEURS DE LA SESSION ARRAY

La méthode **push** peut être utilisée pour insérer une nouvelle valeur dans une valeur de session qui est un tableau.

Par exemple, si la clé **user.teams** contient un tableau de noms d'équipes, vous pouvez pousser une nouvelle valeur sur le tableau comme suit :

```
$request->session()->push('user.teams', 'developers');
```

RÉCUPÉRATION ET SUPPRESSION D'UN ÉLÉMENT

La méthode **pull** permet de récupérer et de supprimer un élément de la session en une seule instruction :

```
$value = $request->session()->pull('key', 'default');
```

INCRÉMENTATION ET DÉCRÉMENTATION DES VALEURS DE LA SESSION

Si vos données de session contiennent un nombre **entier** que vous souhaitez **incrémenter** ou **décrémenter**, vous pouvez utiliser les méthodes **increment** et **decrement**:

```
$request->session()->increment('count');
```

```
$request->session()->increment('count', $incrementBy = 2);
```

```
$request->session()->decrement('count');
```

```
$request->session()->decrement('count', $decrementBy = 2);
```

DONNÉES FLASH

Il peut arriver que vous souhaitiez stocker des éléments dans la session pour la prochaine requête. Vous pouvez le faire en utilisant la méthode **flash**. Les données stockées dans la session à l'aide de cette méthode seront disponibles immédiatement et pendant la requête HTTP suivante. Après la requête HTTP suivante, les données flashées seront supprimées. Les données flash sont principalement utiles pour les messages d'état de courte durée :

```
$request->session()->flash('status', 'La tâche est réussie!');
```

DONNÉES FLASH

Si vous avez besoin de conserver vos données flash pour plusieurs requêtes, vous pouvez utiliser la méthode **reflash**, qui conservera toutes les données flash pour une requête supplémentaire. Si vous avez seulement besoin de conserver des données flash spécifiques, vous pouvez utiliser la méthode **keep** :

```
$request->session()->reflash();
```

```
$request->session()->keep(['username', 'email']);
```


SUPPRESSION DES DONNÉES

La méthode **forget** permet de supprimer un élément de données de la session. Si vous souhaitez supprimer toutes les données de la session, vous pouvez utiliser la méthode **flush** :

```
// Oublier une seule clé...  
$request->session()->forget('name');  
  
// Oubliez les clés multiples...  
$request->session()->forget(['name', 'status']);  
  
$request->session()->flush();
```

RÉGÉNÉRATION DE L'ID DE SESSION

La régénération de l'ID de session est souvent effectuée afin d'empêcher les utilisateurs malveillants d'exploiter une attaque par fixation de session sur votre application.

Laravel régénère automatiquement l'ID de session pendant l'authentification; toutefois, si vous devez régénérer manuellement l'ID de session, vous pouvez utiliser la méthode **regenerate** :

```
$request->session()->regenerate();
```

RÉGÉNÉRATION DE L'ID DE SESSION

Si vous devez régénérer l'ID de session et supprimer toutes les données de la session en une seule instruction, vous pouvez utiliser la méthode **invalidate** :

```
$request->session()->invalidate();
```